TREE ALGORITHMS

DSA Roadmap in Java

A structured guide from tree fundamentals to advanced tree data structures, with interview-focused algorithms, learning order, and complexity notes.

Learning stage	Main focus
1. Fundamentals	Tree traversals and recursion
2. Core problems	Depth, diameter, balance, paths, symmetry
3. BST	Search, insert, delete, validation, kth element
4. Construction	Build from traversals; serialize and deserialize
5. Advanced	Tree DP, tries, segment trees, binary lifting

How to use this roadmap: Learn each algorithm's purpose, implement it in Java, dry-run a small example, and explain its time and space complexity before moving on.

1. Tree Traversal Algorithms

Traversal is the foundation of tree problem-solving. The order determines when each node is visited.

Algorithm	Order / method	Common uses
Preorder DFS	Root → Left → Right	Copying trees, serialization, construction
Inorder DFS	Left → Root → Right	Sorted output from a BST
Postorder DFS	Left → Right → Root	Subtree calculations, deletion, tree DP
Level-order BFS	Level by level using a queue	Level averages, zigzag, side views, minimum depth
Iterative traversals	Use stack(s)	Avoid recursive call stack
Morris traversal	Temporary pointer threading	Inorder/preorder with O(1) auxiliary space, excluding output

2. Basic Binary Tree Algorithms

- **Count nodes / leaf nodes:** Count all nodes or nodes with no children.
- **Maximum depth / height:** Find the longest root-to-leaf path.
- **Minimum depth:** Find the shortest root-to-leaf path.
- **Search:** Find a target in a general binary tree.
- **Invert / mirror:** Swap the left and right subtrees.
- **Same tree / symmetric tree:** Compare structure and values, or check mirror symmetry.
- **Balanced tree:** Check whether subtree heights differ by at most one at every node.
- **Diameter:** Find the longest path between any two nodes.
- **Subtree check:** Determine whether one tree appears within another.
- **Path sum / root-to-leaf paths:** Find paths meeting a target or generate all root-to-leaf paths.
- **Sum, minimum, maximum:** Aggregate values across the tree.

3. Recursion and Divide-and-Conquer

Many tree algorithms solve left and right subtrees independently, then combine the results at the current node.

Reusable Java template — maximum depth

```
class Solution {
public int maxDepth(TreeNode root) {
if (root == null) return 0; // base case

int left = maxDepth(root.left);
int right = maxDepth(root.right);

return 1 + Math.max(left, right);
}
}
```

Related algorithms: diameter, balanced tree, maximum path sum, lowest common ancestor (LCA), path sum, tree construction from traversals, and serialization/deserialization.

4. Binary Search Tree (BST) Algorithms

A BST orders values so left-subtree values are smaller and right-subtree values are larger, assuming distinct values. Balanced BST operations can take O(log n); an unbalanced BST can take O(n).

- Search, insert, and delete a node
- Validate a BST
- Find minimum and maximum
- Find kth smallest / kth largest
- Find predecessor and successor
- Lowest Common Ancestor in a BST
- Convert sorted array to balanced BST
- Recover a BST with swapped nodes
- Find modes in a BST
- Implement a BST iterator
- Trim a BST to a range
- Convert to a greater-sum tree

5. Advanced Traversal and View Algorithms

- Zigzag level-order traversal
- Right-side view and left-side view
- Vertical order traversal, top view, and bottom view
- Boundary traversal and diagonal traversal
- Maximum width of a binary tree
- Nodes at distance K and path between two nodes
- Burn a binary tree (multi-source BFS after locating the start)
- Flatten binary tree to linked list
- Populate next-right pointers
- Check whether nodes are cousins

6. Construction and Transformation

- Build from preorder + inorder
- Build from inorder + postorder
- Build from preorder + postorder when constraints make the tree identifiable
- Construct a BST from preorder
- Construct a tree from level-order traversal
- Serialize and deserialize a binary tree
- Flatten a tree; merge two trees; clone a tree
- Convert a binary tree to a doubly linked list
- Recover a tree from traversal encodings

7. Tree Dynamic Programming (Tree DP)

- Maximum path sum and diameter
- House Robber III

- Maximum independent set on a tree
- Minimum vertex cover on a tree
- Count paths with a given sum
- Longest zigzag path
- Maximum sum of non-adjacent tree nodes
- Minimum cameras to monitor a binary tree
- Rerooting DP (advanced)

8. N-ary Tree Algorithms

- N-ary preorder, postorder, and level-order traversal
- Maximum depth of an N-ary tree
- Level averages and per-level aggregates
- Serialize and deserialize an N-ary tree

9. Advanced Tree Data Structures and Algorithms

Structure / algorithm	Typical use
Trie / Prefix Tree	Prefix search, dictionaries, word lookup
Segment Tree	Range queries and updates
Lazy Propagation	Efficient range updates in a segment tree
Fenwick Tree (BIT)	Prefix sums and point updates
Binary Lifting	Fast ancestor and LCA queries
Euler Tour	Represent subtrees as traversal intervals
Heavy-Light Decomposition	Turn tree path queries into range queries
Centroid Decomposition	Divide-and-conquer on trees
Rerooting DP	Compute an answer for every possible root
Persistent Segment Tree	Maintain versions of range-query structures
Treap / AVL / Red-Black Tree	Balanced search-tree operations

10. Recommended Learning Order

Stage	Learn these topics
1 — Fundamentals	Preorder, inorder, postorder, level-order, recursion
2 — Basic problems	Depth, invert, same tree, symmetry, path sum
3 — Core recursion	Diameter, balanced tree, subtree, LCA
4 — BST	Search, insert, delete, validate BST, kth smallest
5 — Traversal patterns	Zigzag, side views, vertical order, distance K
6 — Construction	Build from traversals, serialization, flattening
7 — Advanced	Tree DP, tries, binary lifting, segment tree, rerooting

11. Complexity Cheat Sheet

Operation	Typical complexity	Notes
Full traversal	O(n) time	Every node visited once
Recursive DFS stack	O(h) auxiliary space	h is tree height; worst case O(n)
BFS queue	O(n) auxiliary space worst case	Can hold an entire level
Balanced BST search	O(log n)	Assumes tree remains balanced
Unbalanced BST search	O(n) worst case	Can degenerate into a chain
Build from traversals + HashMap	O(n) expected time	O(n) map plus recursion/output

12. Practice Checklist

Check off a stage only when you can explain the approach, code it without copying, dry-run it, and state its complexity.

- **Stage 1:** Four traversals, recursion, base cases
- **Stage 2:** Maximum/minimum depth, invert tree, same tree, symmetry
- **Stage 3:** Diameter, balanced tree, path sum, subtree, LCA
- **Stage 4:** BST search, insert, delete, validation, kth smallest
- **Stage 5:** Zigzag, right-side view, vertical order, distance K
- **Stage 6:** Construct from traversals, serialize/deserialize, flatten
- **Stage 7:** Tree DP, trie, binary lifting, segment tree, rerooting

13. Interview Preparation Priorities

Master these first: preorder/inorder/postorder, BFS, recursion, maximum depth, diameter, balanced tree, path sum, LCA, BST validation, kth smallest in BST, and constructing a tree from traversals.

Questions to ask for every tree problem:

1. What information does the current node need from its children?
2. What is the base case?
3. Should I use DFS (recursion/stack) or BFS (queue)?
4. Can I divide the problem into left and right subtrees?
5. What are the time and auxiliary-space complexities?

Java reminder: LeetCode usually provides TreeNode for binary-tree problems. Submit the requested class and method signature; do not add a duplicate TreeNode class unless writing a standalone program.